

ChatGPT

Matrix vs Tensor

diff between matrix and tensors	Show more	Show less		2
describe the image	Show more	Show less		6
diff bwt feature tensor and adjacency tensor	Show more	Show less		11
create visulisation for tensors, tensor matrix,Supra-Adjacency Matrix	Show more	Show less		18
what does a 2d tensor represnet	Show more	Show less		18
how many dimensaional tensor required to represent interlayer edges in multilayer graph	Show more	Show less		22



diff between matrix and tensors

A **matrix** is actually a special case of a **tensor**.
The main difference is the **number of dimensions (axes)** they have.

1. Matrix

A matrix is a **2-dimensional array** of numbers.

It has:

- Rows
- Columns

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

This matrix has:

- 2 rows
- 3 columns
- Shape = 2×3

You can think of a matrix like a table or spreadsheet.

2. Tensor

A tensor is a **generalized multi-dimensional array**.

It can have:

- 0 dimensions
- 1 dimension
- 2 dimensions
- 3 dimensions
- n dimensions

So tensors generalize:

Structure	Dimensions
Scalar	0D tensor

Structure	Dimensions
Vector	1D tensor
Matrix	2D tensor
3D Array	3D tensor
nD Array	nD tensor

Tensor Hierarchy

Scalar $\subset$ Vector $\subset$ Matrix $\subset$ Tensor

Examples

Scalar (0D Tensor)

5

Just one number.

Vector (1D Tensor)

[2, 4, 6]

Only one axis.

Shape:

(3)

Matrix (2D Tensor)

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Shape:

(2, 2)

3D Tensor

Imagine multiple matrices stacked together.

Shape:

$$(3, 2, 2)$$

Meaning:

- 3 matrices
- each matrix is 2×2

Visualization:

$$T[0] = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$T[1] = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$T[2] = \begin{bmatrix} 9 & 10 \\ 11 & 12 \end{bmatrix}$$

Key Difference

Feature	Matrix	Tensor
Dimensions	Always 2D	Can be any dimension
Representation	Rows and columns	Multi-axis structure
Generality	Specific case	Generalized concept
Example Shape	(3,4)	(3,4,5,6...)
Used in	Linear algebra	AI, Deep Learning, Physics

In Machine Learning / Deep Learning

Tensors are used because data is often multi-dimensional.

Examples:

Data	Tensor Shape
Grayscale image	(height, width)
RGB image	(height, width, channels)
Video	(frames, height, width, channels)
Batch of images	(batch, height, width, channels)

Example:

A colored image:

$$(224, 224, 3)$$

because:

- height = 224
- width = 224
- RGB channels = 3

Mathematical View

In mathematics and physics, tensors are more than just arrays.

They represent relationships between vectors, scalars, and other tensors and obey transformation rules under coordinate changes.

Examples in physics:

- Stress tensor
- Inertia tensor
- Electromagnetic tensor

But in deep learning, tensor usually means:

Multi-dimensional numerical array.

Simple Analogy

- Scalar → single value
- Vector → list
- Matrix → table
- Tensor → cube/hypercube of data

Summary

Concept	Meaning
Matrix	2D tensor
Tensor	Generalized multidimensional array
Every matrix is a tensor	Yes
Every tensor is a matrix	No

So:

Matrix is a special case of Tensor

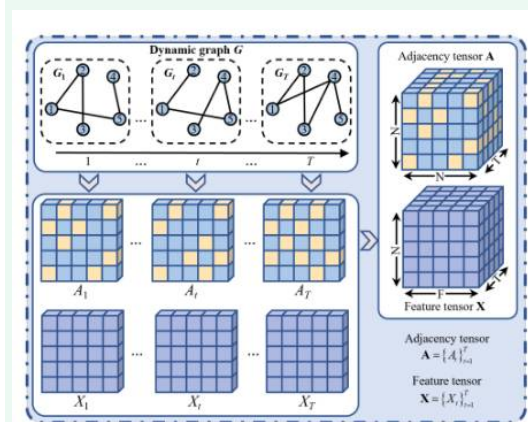


Fig. 1. Third-order tensor form of a dynamic graph.

describe the image

This figure shows how a **dynamic graph (time-varying network)** is represented using **third-order tensors**.

Overall Idea

The image explains:

A graph changes over time, and instead of storing separate adjacency matrices for each timestamp, we stack them together into a 3D tensor.

It also shows how **node features** are stored similarly.

1. Top Left: Dynamic Graph G

The top section shows a graph evolving over time.

There are multiple snapshots:

$$G_1, G_t, G_T$$

where:

- G_1 = graph at time step 1
- G_t = graph at intermediate time t
- G_T = graph at final time T

Nodes

The graph has nodes:

1, 2, 3, 4, 5

Dynamic Nature

Edges change over time.

For example:

- Some connections appear/disappear
- Network structure evolves

This is why it is called a:

Dynamic Graph

instead of a static graph.

Examples:

- Social networks changing daily
 - Traffic networks changing hourly
 - Communication networks changing over time
-

2. Middle Left: Adjacency Matrices Over Time

For every graph snapshot:

$$G_t$$

there is an adjacency matrix:

$$A_t$$

Shown as:

$$A_1, A_t, A_T$$

Each adjacency matrix stores node connectivity at that specific time.

If:

$$(A_t)_{ij} = 1$$

then node i connects to node j at time t .

Color Meaning

Colored squares indicate:

- existence/strength of edges

Blue/light cells:

- edge present or weighted connection

Empty cells:

- no edge

3. Bottom Left: Feature Matrices X_t

Below adjacency matrices are feature matrices:

$$X_1, X_t, X_T$$

These store node attributes/features over time.

What are Node Features?

Examples:

Network	Node Features
Social network	age, activity, followers
Traffic network	speed, congestion
IoT network	sensor readings
Brain network	signal intensity

If:

- N = number of nodes
- F = number of features

then:

$$X_t \in \mathbb{R}^{N \times F}$$

4. Right Side: Tensor Representation

The separate matrices are stacked into tensors.

This is the main idea of the figure.

A. Adjacency Tensor

Top-right cube:

$$\mathbf{A} = \{A_t\}_{t=1}^T$$

Dimensions

The tensor has dimensions:

$$N \times N \times T$$

where:

- first N = source node
 - second N = destination node
 - T = time
-

Meaning

$$\mathcal{A}_{ijt}$$

means:

connection between node i and node j at time t

Visualization

Think of it as:

Stacking adjacency matrices through time.

Like:

$$A_1 \rightarrow A_2 \rightarrow A_3$$

forming a 3D cube.

B. Feature Tensor

Bottom-right cube:

$$\mathbf{X} = \{X_t\}_{t=1}^T$$

Dimensions

$$N \times F \times T$$

where:

- N = nodes
- F = features
- T = time

Meaning

$$\mathcal{X}_{ift}$$

means:

feature f of node i at time t

Why Tensor Representation?

Instead of processing each timestamp separately:

$$A_1, A_2, \dots, A_T$$

we use one tensor:

$$\mathcal{A}$$

This helps machine learning models learn:

- temporal patterns
- evolving connectivity
- spatiotemporal dependencies

Applications

This representation is heavily used in:

Field	Use
Traffic prediction	Road conditions over time
Social networks	Evolving interactions
Brain networks	Dynamic connectivity
IoT	Sensor streams
Financial systems	Transaction evolution
Graph Neural Networks	Temporal GNNs

Key Insight of the Figure

The figure transforms:

Dynamic graph sequence

$$G_1, G_2, \dots, G_T$$

into:

Tensor representation

$$\mathcal{A} \in \mathbb{R}^{N \times N \times T}$$

and

$$\mathcal{X} \in \mathbb{R}^{N \times F \times T}$$

so deep learning models can process time-evolving graphs efficiently.

Final Intuition

The image shows:

Static Graph

→ one adjacency matrix

Dynamic Graph

→ many adjacency matrices over time

Tensor Representation

→ stack them into a 3D cube for temporal graph learning.



diff bwt feature tensor and adjacency tensor

Difference Between Feature Tensor and Adjacency Tensor

Both are used in dynamic graphs, Graph Neural Networks (GNNs), and temporal networks, but they represent completely different things.

Core Difference

Tensor	Represents
Adjacency Tensor	Graph structure / connectivity
Feature Tensor	Properties/attributes of nodes

1. Adjacency Tensor

The adjacency tensor tells:

“Who is connected to whom over time/layers?”

It stores **edges**.

Mathematical Form

$$\mathcal{A} \in \mathbb{R}^{N \times N \times T}$$

where:

- N = number of nodes
- T = time steps

Element Meaning

$$\mathcal{A}_{ijt}$$

means:

Connection between node i and node j at time t

Example

Suppose:

- Node 1 connected to node 3 at time 2

Then:

$$\mathcal{A}_{132} = 1$$

What It Encodes

- graph topology
- edge existence
- edge weights
- network dynamics

Visualization

Each slice:

$$A_t$$

is an adjacency matrix at time t .

Stacking them creates a cube.

2. Feature Tensor

The feature tensor tells:

“What properties does each node have over time?”

It stores **node attributes/features**.

Mathematical Form

$$\mathcal{X} \in \mathbb{R}^{N \times F \times T}$$

where:

- N = nodes
- F = features
- T = time

Element Meaning

$$\mathcal{X}_{ift}$$

means:

Feature f of node i at time t

Example

Suppose:

- Node 2 temperature = 45°C at time 3

Then:

$$\mathcal{X}_{2,\text{temp},3} = 45$$

What It Encodes

- sensor readings
- node embeddings
- user activity
- traffic speed
- node states
- biological measurements

Simple Analogy

Imagine a social network.

Adjacency Tensor

Stores:

Person A	Person B	Time	Connected?
Alice	Bob	Day 1	Yes

This is relationship information.

Feature Tensor

Stores:

Person	Feature	Time	Value
Alice	Followers	Day 1	500

This is attribute information.

Visual Interpretation

Adjacency Tensor

$$N \times N \times T$$

Axes:

- node
- node
- time

Represents:

Connections

Feature Tensor

$$N \times F \times T$$

Axes:

- node
- feature
- time

Represents:

Node properties

Example in Traffic Network

Suppose:

- Nodes = road sensors

Adjacency Tensor

Stores:

Which roads are connected at time t .

Feature Tensor

Stores:

- speed
- congestion
- temperature
- traffic density

for each sensor over time.

Example in IoT

Adjacency Tensor

Device communication structure.

Who talks to whom.

Feature Tensor

Sensor values:

- temperature
 - humidity
 - battery level
-

In Graph Neural Networks

Both tensors work together.

Adjacency Tensor

Controls:

Message Passing

Which nodes exchange information.

Feature Tensor

Provides:

Information Content

What information each node carries.

Formula Perspective

In GNNs:

$$H^{(l+1)} = f(A, H^{(l)})$$

where:

- A = adjacency structure
- H = node features

Dynamic Graph Version

Now:

$$\mathcal{A}, \mathcal{X}$$

evolve over time.

Key Difference Table

Aspect	Adjacency Tensor	Feature Tensor
Stores	Edges/connections	Node attributes
Meaning	Network structure	Node information
Shape	$N \times N \times T$	$N \times F \times T$
Element	connection strength	feature value
Used for	topology/message passing	learning representations
Sparse/Dense	usually sparse	often dense

Final Intuition

Adjacency Tensor

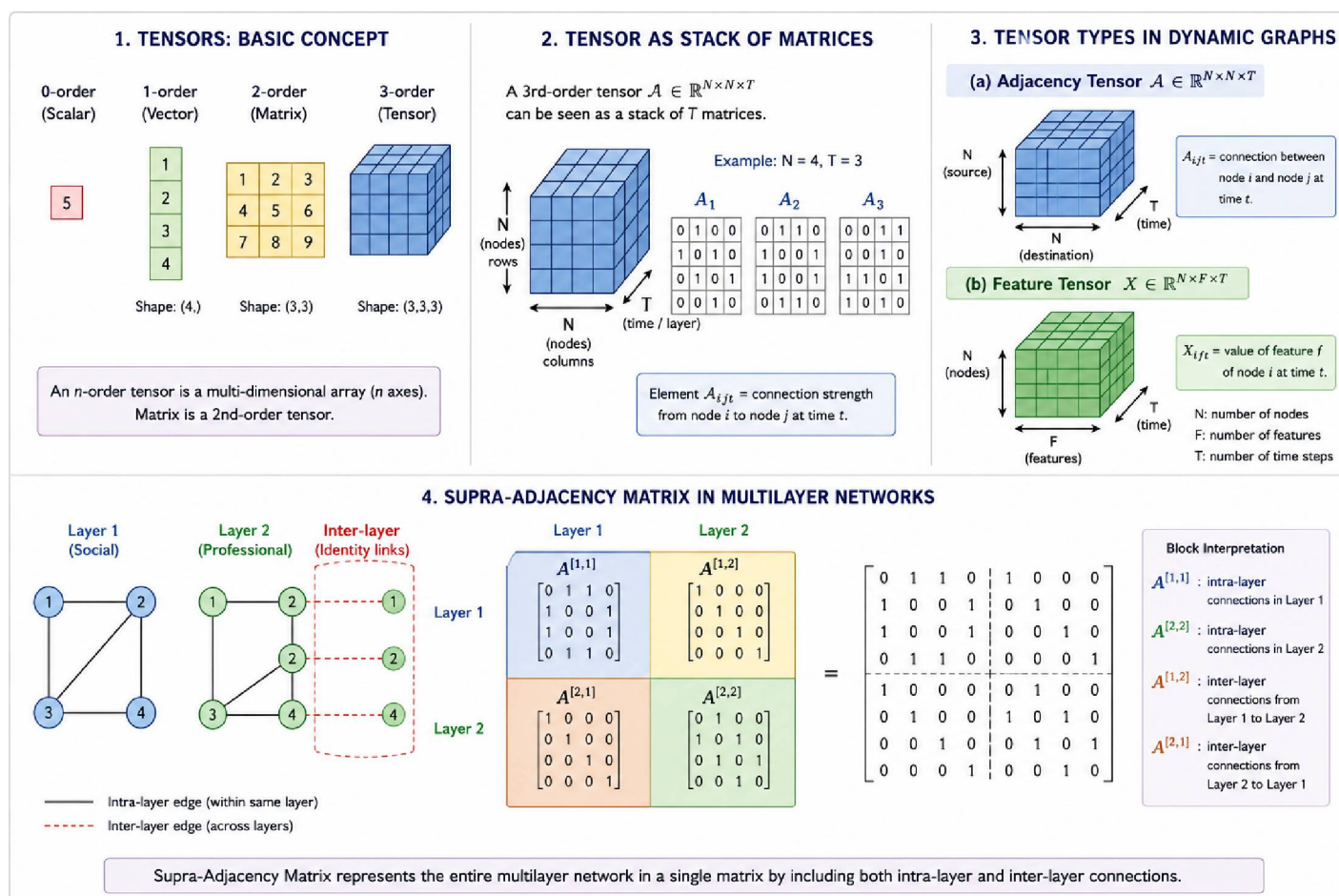
“Who interacts with whom?”

Feature Tensor

"What characteristics does each node have?"



create visulisation for tensors, tensor matrix,Supra-Adjacency Matrix



Structure of a 2D Tensor

A 2D tensor has:

- rows
- columns

Shape:

$$(m, n)$$

where:

- m = number of rows
- n = number of columns

Example

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Shape:

$$(2, 3)$$

Meaning:

- 2 rows
- 3 columns

What Does It Represent?

A 2D tensor can represent many things depending on context.

1. Tabular Data

Example:

Student	Math	Science
A	90	85
B	70	95

Tensor:

$$\begin{bmatrix} 90 & 85 \\ 70 & 95 \end{bmatrix}$$

Rows = students

Columns = subjects

2. Graph Connectivity (Adjacency Matrix)

In graphs:

$$A_{ij}$$

represents:

connection between node i and node j

Example:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

3. Image (Grayscale)

A grayscale image is a 2D tensor.

Each value = pixel intensity.

Shape:

$$(height, width)$$

Example:

$$28 \times 28$$

for MNIST handwritten digits.

4. Feature Matrix in ML

Suppose:

- rows = samples
- columns = features

Then:

$$X \in \mathbb{R}^{N \times F}$$

Example:

Person	Age	Salary
A	25	50K
B	30	80K

Mathematical Interpretation

A 2D tensor maps:

$$T(i, j)$$

meaning:

value at row i , column j

Tensor Order Perspective

Tensor Type	Dimensions	Example
Scalar	0D	5
Vector	1D	[1, 2, 3]
Matrix	2D	table/grid
3D Tensor	3D	stack of matrices

Visual Intuition

A 2D tensor is like:

- spreadsheet
- chessboard
- image grid
- graph adjacency table

Important Point

A 2D tensor itself does not have a fixed meaning.

Its meaning depends on:

- what rows represent
- what columns represent

In Deep Learning

A 2D tensor often represents:

$$(batch_size, features)$$

Example:

$$(32, 128)$$

means:

- 32 samples
- 128 features each

Final Intuition

A 2D tensor represents:

Data organized across two independent axes.

Usually:

$$\text{rows} \times \text{columns}$$

or:

$$\text{objects} \times \text{attributes}$$

or:

$$\text{node} \times \text{node}$$

depending on the application.



how many dimensional tensor required to represent interlayer edges in multilayer graph

To represent **interlayer edges** in a multilayer graph properly, you typically need a:

4-dimensional adjacency tensor

Why?

In an ordinary graph, an edge is:

$$(i, j)$$

So a 2D adjacency matrix is enough:

$$A_{ij}$$

But in a multilayer graph, each node also belongs to a layer.

A node becomes:

$$(v_i, \alpha)$$

where:

- i = node index
- α = layer index

Now an edge must specify:

1. source node
2. destination node
3. source layer
4. destination layer

So the adjacency becomes:

$$\mathcal{A}_{ij\alpha\beta}$$

This requires:

4 dimensions

Tensor Dimensions

$$\mathcal{A} \in \mathbb{R}^{N \times N \times L \times L}$$

where:

- N = number of nodes
- L = number of layers

Meaning of Each Axis

Dimension	Meaning
i	source node
j	destination node

Dimension	Meaning
α	source layer
β	destination layer

Interpretation

$$\mathcal{A}_{ij\alpha\beta} = 1$$

means:

Node i in layer α connects to node j in layer β

Special Cases

1. Intra-layer Edge

Same layer:

$$\alpha = \beta$$

Example:

$$\mathcal{A}_{23,1,1}$$

means:

Node 2 connected to node 3 inside layer 1.

2. Interlayer Edge

Different layers:

$$\alpha \neq \beta$$

Example:

$$\mathcal{A}_{22,1,2}$$

means:

Node 2 in layer 1 connected to node 2 in layer 2.

Visualization

Think of it like:

Instead of one adjacency matrix:

$$A_{ij}$$

you now have a matrix of matrices.

Each block:

$$A^{[\alpha, \beta]}$$

represents connections:

from layer α to layer β .

Relation to Supra-Adjacency Matrix

The 4D tensor can be flattened into a large 2D matrix called the:

Supra-Adjacency Matrix

Example:

$$\mathbf{M} = \begin{bmatrix} A^{[1,1]} & A^{[1,2]} \\ A^{[2,1]} & A^{[2,2]} \end{bmatrix}$$

where:

- diagonal blocks = intra-layer edges
- off-diagonal blocks = interlayer edges

Why 3D Is Not Enough

A 3D tensor:

$$\mathcal{A}_{ijt}$$

can represent:

- dynamic graphs over time

OR

$$\mathcal{A}_{ijl}$$

can represent:

- multiple relation types/layers

BUT it cannot independently encode:

- source layer
- destination layer

simultaneously.

That requires another dimension.

General Rule

Graph Type	Tensor Order
Static graph	2D
Dynamic graph	3D
Multilayer graph with interlayer edges	4D
Dynamic multilayer graph	5D or more

Dynamic Multilayer Example

If time is also added:

$$\mathcal{A}_{ij\alpha\beta t}$$

Now dimensions are:

- node
- node
- source layer
- destination layer
- time

So this becomes:

5D tensor

Final Answer

To fully represent interlayer edges in a multilayer network, you generally need:

a 4-dimensional adjacency tensor $\mathcal{A}_{ij\alpha\beta}$

because an edge must encode:

- source node

- destination node
- source layer
- destination layer